# ALGORITHMS & COMPLEXITY
# LECTURE 8

We address now DIVIDE & CONQUER ALGORITHMS

Example:


finding maximal element of $L(2n)$

a) $\max L_1(n)$ find
b) $\max L_2(n)$ find
c) $\max L(2n) = \max\{m_1, m_2\}$

} & continue deep down

## DIVIDE & CONQUER PARADIGM

- initial problem is divided into 2 (or more subparts)
- each subpart is solved separately (we can use here one processor or can execute the above tasks parallely with multiple processors)
- next we merge the results from the subparts
- we nest this approach.

# ALGORITHMS & COMPLEXITY
# LECTURE 8

Let $T(n)$ denotes the time to execute a particular task (or number of operations, or size of consumed memory) for a given algorithm where $n$ denotes e.g length of list, size of task to be performed etc.

For $n = 2^k$ (& two subproblems)

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + f(n)$$

- $T\left(\frac{n}{2}\right)$: time to execute the algorithm for each part separately
- $f(n)$: time needed to merge the results

$$\Downarrow$$

$$\boxed{T(n) = 2\,T\left(\frac{n}{2}\right) + f(n)} \quad (*)$$

this is the recurrent formula example for D & Q paradigm.
divide conquer

We formulate now th. concerning (*)

– it can be generalized to $T_n = 3T\left(\frac{n}{3}\right) + f(n)$ & more

# ALGORITHMS & COMPLEXITY
# LECTURE 8

Theorem:

Let $S_n$ be the sequence satisfying:

$$(*)\quad \begin{cases} S_{2n} = 2S_n + f(n) \\ S_1 \text{ given} \end{cases}$$

then for $n = 2^m$ $m \in \mathbb{N}$

$$\boxed{S_{2^m=n} = 2^m \left[ S_1 + \frac{1}{2} \sum_{i=0}^{m-1} \frac{f(2^i)}{2^i} \right]} \quad (**)$$

Note that still in (**) we need to calculate $f(2^0), f(2^1), f(2^2), \ldots, f(2^{m-1})$ values which in $S_{2^m}$ is also happening $S_{2^{m-1}}, S_{2^{m-2}}, S_{2^1}, S_{2^0}$. But (**) can be called semi-explicit as it helps for special cases of $f(n) = A + Bn$ (common in computer science) to find a full explicit form of $S_{2^m}$ & thus assess

$$S_{2^m=n} = O(?)$$

# ALGORITHMS & COMPLEXITY
# LECTURE 8

We shall prove now (**) by Mathematical Induction (running over $m$ where $n=2^m$)

I) (Initial step)

by (*) $S_1 = 1$ $\quad m=0$ but in (**) $m-1=-1$ & no sum

So we need an induction from $m=1$

by (*) $S_2 = \dfrac{2\cdot S_1 + f(1)}{}$

in (**) $S_2 = 2^1\left[S_1 + \frac{1}{2}\sum_{i=0}^{0}\frac{f(2^i)}{2^i}\right]$

$= 2S_1 + \frac{1}{2}\cdot 2\frac{f(0)}{2^0} = \underline{2S_1 + f(1)}$

so ok.

II Assume now $P(m)$: $S_{2^m} = 2^m\left[S_1 + \frac{1}{2}\sum_{i=0}^{m-1}\frac{f(2^i)}{2^i}\right]$ holds

$\Downarrow$ ?

$P(m+1)$: $S_{2^{m+1}} = 2^{m+1}\left[S_1 + \frac{1}{2}\sum_{i=0}^{m}\frac{f(2^i)}{2^i}\right]$

$S_{2^{m+1}} = S_{2\cdot 2^m} \overset{*}{=} 2S_{2^m} + f(2^m)$

$\overset{P(m)}{=} 2\left[2^m\left[S_1 + \frac{1}{2}\sum_{i=0}^{m-1}\frac{f(2^i)}{2^i}\right]\right] + f(2^m)$

Thus

$$S_{2^{m+1}} = 2^{m+1}\left[S_1 + \frac{1}{2}\sum_{i=0}^{m-1}\frac{f(2^i)}{2^i}\right] + \frac{f(2^m)}{2\cdot 2^m}2^{m+1}$$

$$= 2^{m+1}\left[S_1 + \frac{1}{2}\left(\sum_{i=0}^{m-1}\frac{f(2^i)}{2^i} + \frac{f(2^m)}{2}\right)\right]$$

$$= 2^{m+1}\left[S_1 + \frac{1}{2}\sum_{i=0}^{m}\frac{f(2^i)}{2^i}\right]$$

$$= \underbrace{2^{m+1}\left[S_1 + \frac{1}{2}\sum_{i=0}^{(m+1)-1}\frac{f(2^i)}{2^i}\right]}_{P(m+1)}$$

Thus by the principle of mathematical induction

$$P(m) \text{ is true } \forall\, m \geq 1.$$

Let us apply (**) to $f(n) = A + Bn$. ← c)

AND THEN TO: ← a) & b) or c) common cases for algorithms in Comp. Sc.

a) $A \neq 0$ & $B = 0$ ←

b) $A = 0$ & $B \neq 0$ ←

# ALGORITHMS & COMPLEXITY
# LECTURE 8

$f(n) = A + Bn$

$$S_{2^m} = 2^m \left[ S_1 + \frac{1}{2} \sum_{i=0}^{m-1} \frac{A + 2^i B}{2^i} \right]$$

$$= 2^m \left[ S_1 + \frac{1}{2} A \sum_{i=0}^{m-1} \frac{1}{2^i} + \frac{B}{2} \sum_{i=0}^{m-1} 1 \right]$$

$$= 2^m S_1 + \frac{1}{2} 2^m \frac{1 - \left(\frac{1}{2}\right)^m}{1 - \frac{1}{2}} A + 2^m \frac{B}{2} \underbrace{(1+1+\ldots+1)}_{m\text{-times}}$$

↑ sum of the first $m$ terms of geometric sequence

$$\frac{1-q^m}{1-q} = 1 + q + \ldots + q^{m-1}$$

$$= 2^m S_1 + 2^m \left(1 - \left(\frac{1}{2}\right)^m\right) A + m 2^m \frac{B}{2}$$

Thus:

$$\boxed{S_{2^m} = 2^m S_1 + (2^m - 1) A + m 2^m \frac{B}{2}}$$

since $n = 2^m$ $\quad m = \lg_2 n$

(***) $$\boxed{S_n = n S_1 + (n-1) A + n \lg_2 n \frac{B}{2}}$$

We consider now special subcase:

$B = 0$ & $A \neq 0$ (with the specific example)

# ALGORITHMS & COMPLEXITY
# LECTURE 8

**Example1:**

DIVIDE & CONQUER for finding a maximal number in the list.

$$f(n) = A + 0 \cdot B = A.$$

↑ time to compare $m_1$ & $m_2$

$\boxed{T(2n) = 2T(n) + A}$ here

By $\left(\begin{smallmatrix}***\\***\end{smallmatrix}\right)$ we have

$$T(n = 2^m) = \underset{\parallel}{2^m}\, S_1 + (\underset{\parallel}{2^m} - 1)A$$

$n$ $\qquad$ $n$

↑ time needed to find a maximal element in List of length 1

$$\boxed{T(n) = n S_1 + (n-1)A}$$

we have n-1 comparisons

$\boxed{T(n) = \Theta(n)}$ ↑ we check n elements in lists of 1-element

Note that if we use a naive algorithm to find a maximal element of the list


take $a_1$ & $a_2$ compare $a_1 \leq a_2$

take $\tilde{a}_1 = \max\{a_1, a_2\}$

take now $a_3$ and compare with $\tilde{a}_1$

$\tilde{a}_2 = \max\{\tilde{a}_1, a_3\}$ ... continue..

# ALGORITHMS & COMPLEXITY
## LECTURE 8

we have for this naive algorithm:

a) $n-1$ comparisons (each with cost A)

b) $n$ checks of each elements

⇓

So algorithm to find $\max\{L(2n)\}$

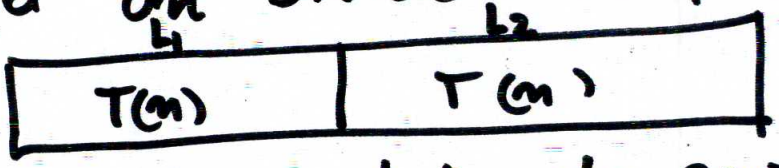
a) find $\max\{L_1(n)\}$ $m_1$

b) find $\max\{L_2(n)\}$ $m_2$

c) $m = \max\{m_1, m_2\}$

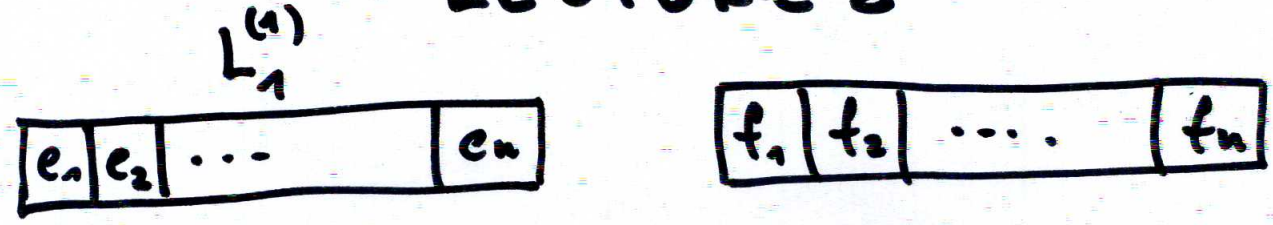
does not give acceleration over the standard sequential algorithm. Only one can get some time execution acceleration if 2 processors are used.

**Example 2:** Sorting the list by using the method MERGE-SORT (based on Divide & Conquer paradigm)




$T(n)$: time needed to sort $L_1(n)$ & $L_2(n)$ with length $n$.

# ALGORITHMS & COMPLEXITY
## LECTURE 8




$$g_1 = \min\{e_1, f_1\} = \min\{L_1^{(1)}, L_2^{(1)}\}$$

$$\begin{cases} L_1^{(2)} := L_1^{(1)} & \text{if } g_1 = f_1 \\ L_2^{(2)} = L_2^{(1)} \setminus \{f_1\} \end{cases}$$

$$\begin{cases} L_1^{(2)} := L_1^{(1)} \setminus \{e_1\} & \text{if } g_1 = e_1 \\ L_2^{(2)} := L_2^{(1)} \end{cases}$$

$$g_2 = \min\{L_1^{(2)}, L_2^{(2)}\}$$

& so on

We have here

$A_1 \cdot n$ comparisons of 2 elements

$A_2 \cdot n$ cost of getting $n$ elements from $L_1$

$A_3 \cdot n$ cost of getting $n$ elements from $L_2$

Total cost $A_1 n + A_2 n + A_3 n = \underbrace{(A_1 + A_2 + A_3)}_{B} \cdot n$

Here $f(n) = Bn$. So for merge-sort

$$\boxed{T(2n) = 2T(n) + Bn}$$

# ALGORITHMS & COMPLEXITY
# LECTURE 8

By $(***)$ we have:

$$\boxed{T(n) = n\,T(1) + \frac{B}{2}\, n \lg_2 n}$$

$$\boxed{T(n) = \Theta(n \lg_2 n)} \quad \begin{matrix} A=0 \\ B\neq 0 \end{matrix}$$

But other algorithms to sort $L(n)$

— bubble sort $\Theta(n^2)$

— insertion sort $\Theta(n^2)$

So even on a single processor merge-sort gives a smaller complexity to sort a list than a bubble sort or insertion sort.

$$\begin{cases} T(n) = 4\,T(\lfloor \frac{n}{2} \rfloor) \\ T(1) = 1 \end{cases} \Rightarrow \text{we showed } T(n) = \Theta(n^2)$$

divide & conquer type